

指令與腳本指南 (Scripting and Commands Guide)

本文件說明 mudclient-rs 客戶端支援的指令輸入與 Lua 腳本功能。

客戶端指令 (Client Commands)

這些指令可以直接在輸入框中使用，以 `#` 或 `/` 開頭。

1. 迴圈執行 (`#loop`)

重複執行指定的指令。

- 語法: `#loop <次數> <指令>`
- 範例:
 - `#loop 3 smile` (執行 smile 3 次)
 - `#loop 5 get all from corpse` (從屍體拿取東西 5 次)

2. 延遲執行 (`#delay`)

在指定的毫秒數後執行指令。

- 語法: `#delay <毫秒> <指令>`
- 範例:
 - `#delay 1000 look` (1 秒後執行 look)
 - `#delay 500 n;e;s` (0.5 秒後執行 n, e, s)

3. 執行 Lua 代碼 (`/lua`)

直接在命令行執行一行 Lua 代碼。

- 語法: `/lua <代碼>`
- 範例:
 - `/lua mud.echo("Hello from Lua!")`
 - `/lua mud.send("say 目前時間: " .. os.date())`

4. 變數操作 (`#var` , `#unvar`)

設定或刪除持久化變數（這些變數可以在觸發器、別名和 Lua 腳本中通過 `variables` 表訪問，或在指令中使用 `$varname`）。

- 設定變數: `#var <名稱> <值>`
 - 範例: `#var target big_monster`
- 刪除變數: `#unvar <名稱>`
 - 範例: `#unvar target`
- 使用變數: 在指令中若是 `$名稱` 會被替換。
 - 範例: `kill $target` (若 target 為 big_monster，則發送 `kill big_monster`)

5. 路徑與移動 (`#path`)

內建的路徑記錄與自動移動功能。

- 語法: #path <子指令>
- 子指令:
 - start / record : 開始記錄移動路徑。
 - stop : 停止記錄。
 - show : 顯示目前記錄的路徑。
 - back : 自動沿著原路返回 (Backtrack)。
 - clear : 清除目前路徑。
 - undo : 刪除上一步記錄。
 - save <名稱> : 將目前路徑儲存到 Profile 中。
 - simplify / optimize : 優化路徑 (合併重複移動)。
 - loop <on|off> : 開啟/關閉迴圈偵測功能。

Lua 腳本 API (Lua Scripting API)

在觸發器 (Triggers)、別名 (Aliases) 的腳本模式中，或使用 `/lua` 時，可以使用 `mud` 物件與客戶端互動。

 **進階閱讀：**關於觸發器與別名的完整使用方式（包含設定格式、正則捕獲、參數替換等），請參考 [觸發器與別名指南 \(Triggers & Aliases\)](#)。

mud 物件函數

函數	說明	範例
<code>mud.send(command)</code>	發送指令到伺服器	<code>mud.send("look")</code>
<code>mud.echo(text)</code>	在主視窗顯示訊息 (不會發送到伺服器)	<code>mud.echo("腳本執行中...")</code>
<code>mud.log(message)</code>	寫入訊息到系統日誌	<code>mud.log("偵測到 Boss 出現")</code>
<code>mud.gag_message()</code>	攔截當前行，不顯示在視窗中 (通常用於觸發器)	<code>mud.gag_message()</code>
<code>mud.window(name, text)</code>	將訊息輸出到指定的子視窗	<code>mud.window("chat", "頻道訊息...")</code>
<code>mud.timer(seconds, code)</code>	設定延遲執行 (單位: 秒)	<code>mud.timer(2.5, "mud.send('heal')")</code>
<code>mud.enable_trigger(name, bool)</code>	啟用或禁用指定名稱的觸發器	<code>mud.enable_trigger("autoloot", false)</code>

變數與表格

- **variables** : 全域變數表 (Table) 。
 - 讀取: `local t = variables["target"]`
 - 寫入: `variables["target"] = "orc"`
- **matches** (或 **captures**) : 觸發器的正則表達式捕獲組 (Captures) 。
 - `captures[1]` 代表第一個括號捕捉到的內容。
- **message** : 當前觸發的原始訊息行。

範例腳本

以下是一個綜合範例，展示如何編寫 Lua 腳本來自動化操作。

```
-- 自動治療腳本範例
-- 假設觸發器捕捉到: "你的生命值還剩下 (100)/500"
-- 正則表達式: 你的生命值還剩下 (\d+)/(\d+)

local current_hp = tonumber(captures[1])
local max_hp = tonumber(captures[2])

if current_hp < (max_hp * 0.3) then
    mud.echo("⚠ 生命危急！自動喝藥水...")
    mud.send("drink health_potion")

    -- 2秒後檢查是否需要再喝
    mud.timer(2.0, [[
        if variables["auto_heal"] == "true" then
            mud.send("score") -- 檢查狀態
        end
    ]])
else
    mud.echo("生命值安全: " .. current_hp)
end
```

相關文件

- [觸發器與別名指南 \(Triggers & Aliases\)](#)
- [腳本開發指南 \(Script Guide\)](#)
- [腳本使用手冊 \(Scripts\)](#)